



Workflow vs AI Agents



Understanding agentic system architectures

Module: Agentic AI • Workflows & Agents

Nunnari Academy

Generative AI & Agentic AI Program

Setting the Stage

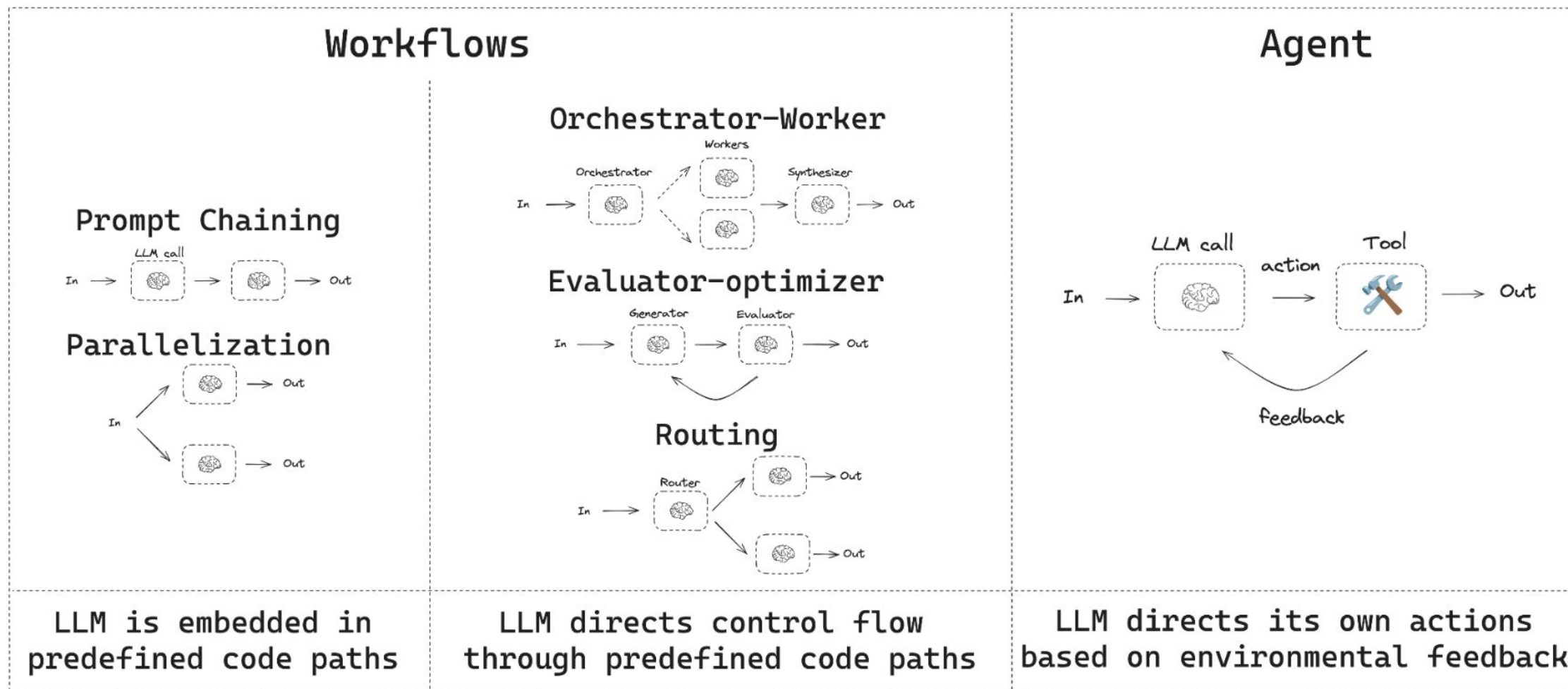


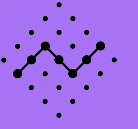
1 What's the difference between an LLM with tools, a workflow, and an AI Agent?

2 Why do they all sound similar to each other?

3 Is there really any major difference between them?

Workflows vs Agents





Augmented LLM

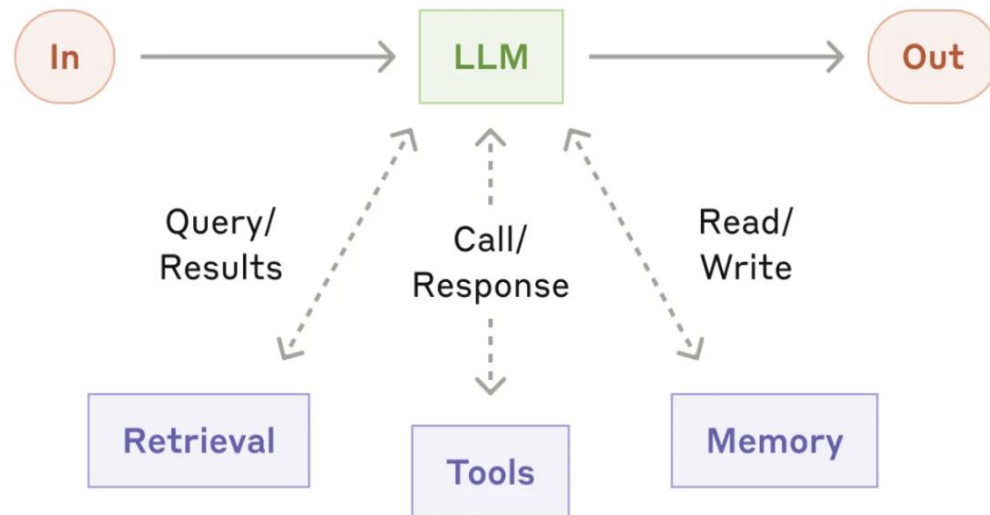




1

Augmented LLM

Foundational building block – An LLM enhanced with retrieval, tools, and memory – the base unit of every agentic system.



What it is

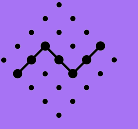
- Modern LLMs actively use these capabilities
- Generate their own search queries
- Select appropriate tools for the task
- Decide what information to retain in memory

Implementation priorities

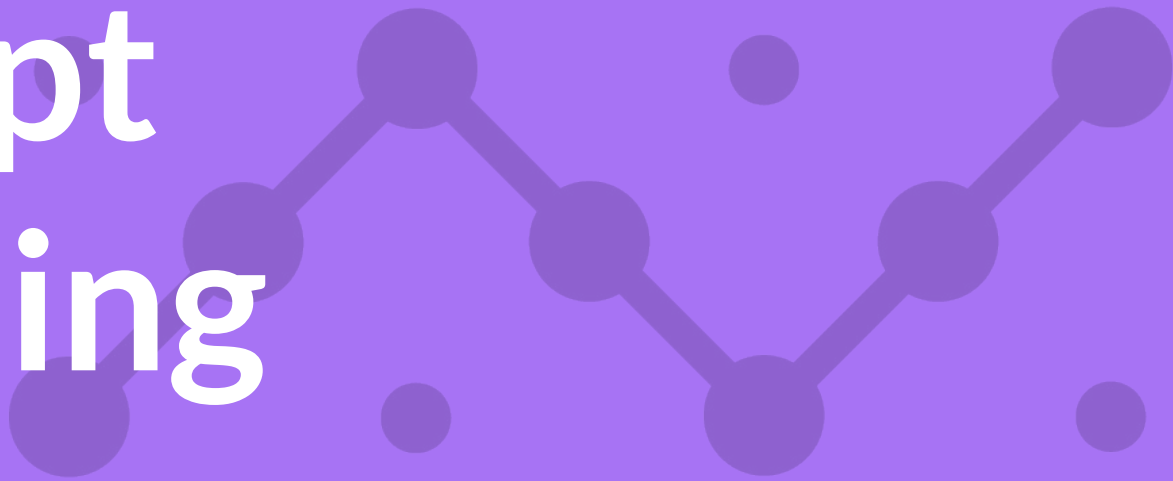
- Tailor capabilities to your specific use case
- Provide an easy, well-documented interface

Note on MCP

- Model Context Protocol enables integration
- Connects to a growing ecosystem of third-party tools
- Simple client implementation



Prompt Chaining





2

Prompt Chaining

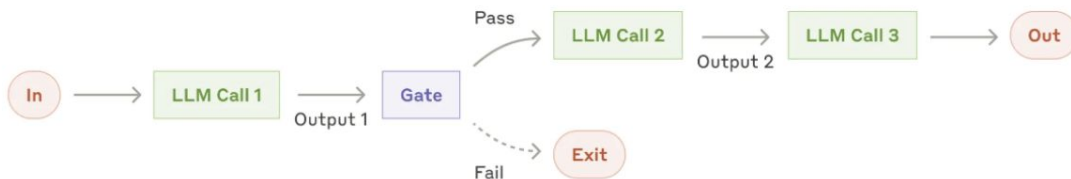
Sequential workflow — Each LLM call processes the output of the previous one — a sequence of steps with optional programmatic gates.

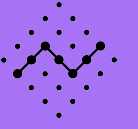
When to use

- Task can be cleanly decomposed into fixed subtasks
- Trade off latency for higher accuracy
- Each step becomes an easier LLM task

Examples

- Topic → make joke → check joke → improve twice
- Generate marketing copy → translate to another language
- Write outline → verify outline → write full document





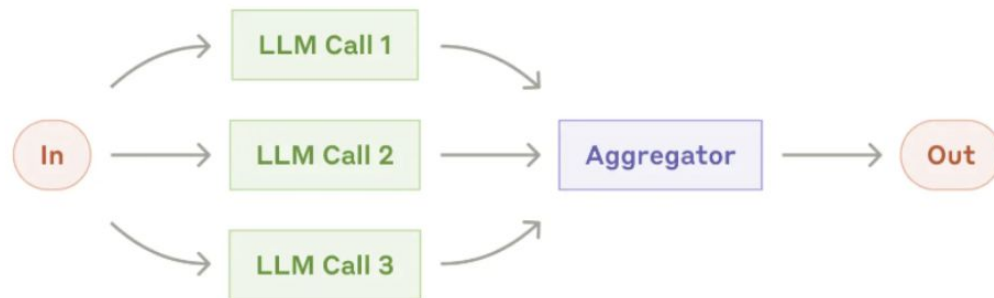
Parallelization



3

Parallelization

Concurrent workflow — LLMs work simultaneously on a task — outputs are aggregated programmatically.



Two variations

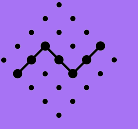
- Sectioning — independent subtasks run in parallel
- Voting — same task run multiple times for diverse outputs

When to use

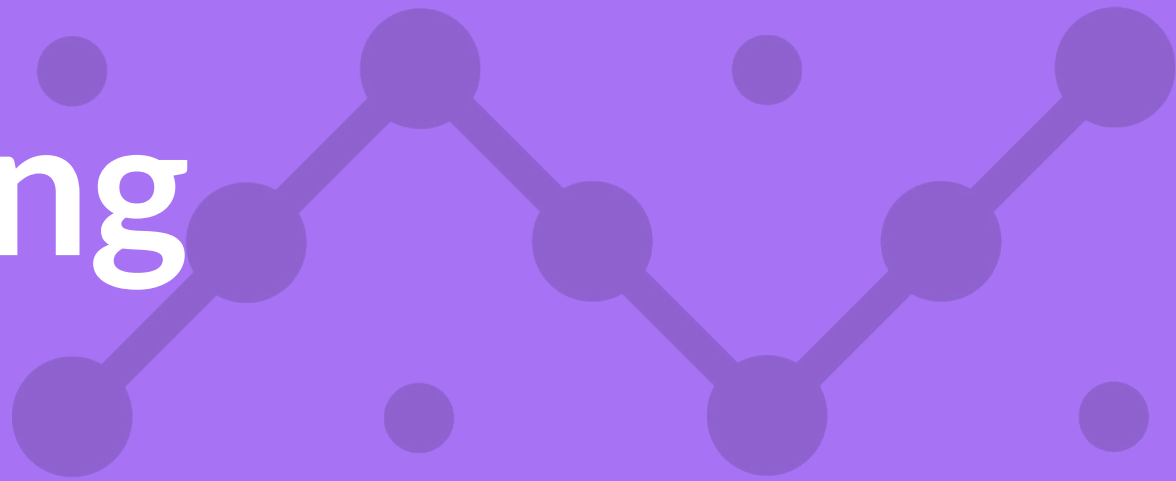
- Speed gains from parallel subtasks
- Multiple perspectives boost confidence

Examples

- Topic → joke, story, and poem in parallel
- Multi-query RAG (multiple perspectives on one question)
- Guardrails: one model handles query, another screens content
- Code vulnerability review with voting prompts



Routing





4

Routing

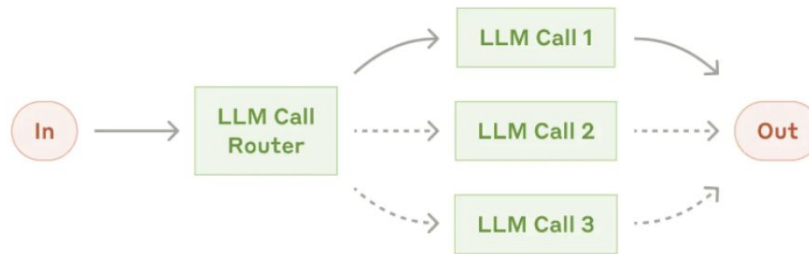
Classification workflow — Classify an input and direct it to a specialized follow-up task.

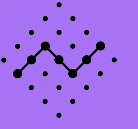
When to use

- Complex tasks with distinct categories
- Categories are better handled separately
- Classification is reliable (LLM or traditional model)

Examples

- Route input between joke, story, and poem generators
- Customer service: general / refund / technical support
- Cost routing: easy → Haiku, hard → Sonnet
- Routing questions to different retrieval systems





Orchestrator Worker



5

Orchestrator-Worker

Dynamic delegation — A central LLM dynamically breaks down tasks, delegates to worker LLMs, and synthesizes their results.

Key difference from parallelization

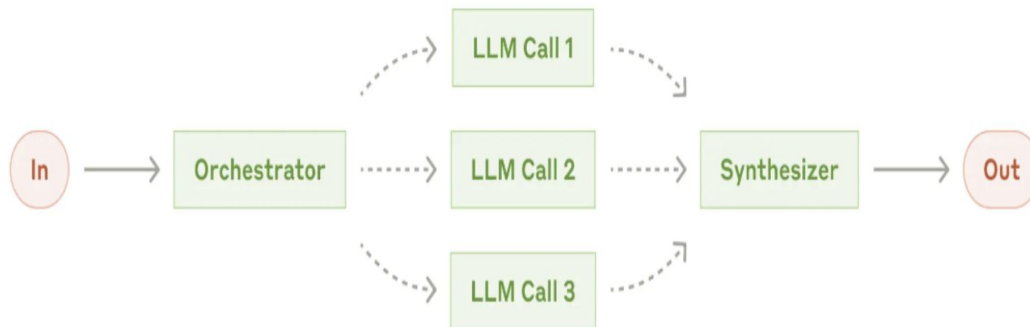
- Subtasks are NOT pre-defined
- The orchestrator decides them based on the input

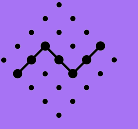
When to use

- Complex tasks where subtasks can't be predicted
- Number/nature of subtasks depends on the input

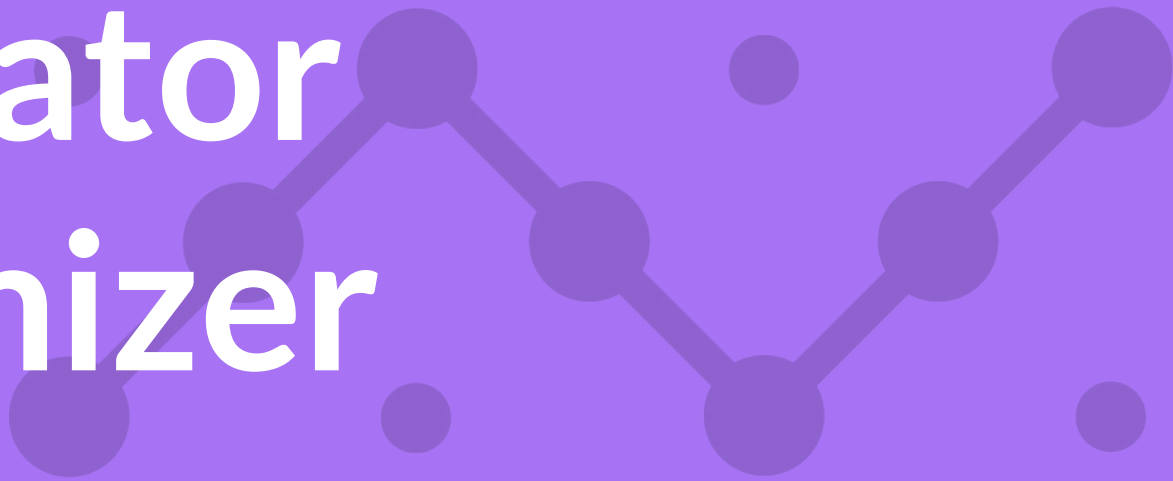
Examples

- Topic → plan report sections → workers write each section
- Coding agents that edit a variable number of files
- Search tasks gathering info from multiple sources





Evaluator Optimizer

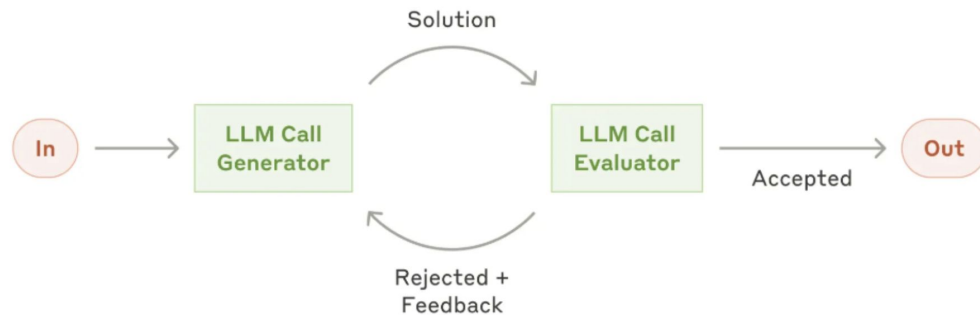




6

Evaluator-Optimizer

Iterative refinement loop – One LLM generates a response while another evaluates and provides feedback – in a loop.



When to use

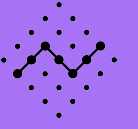
- Clear evaluation criteria exist
- Iterative refinement provides measurable value

Two signs of good fit

- Responses improve when humans articulate feedback
- The LLM can produce that kind of feedback itself

Examples

- Grading RAG responses for hallucinations
- Literary translation with nuance critiques
- Multi-round search where evaluator decides on more searches



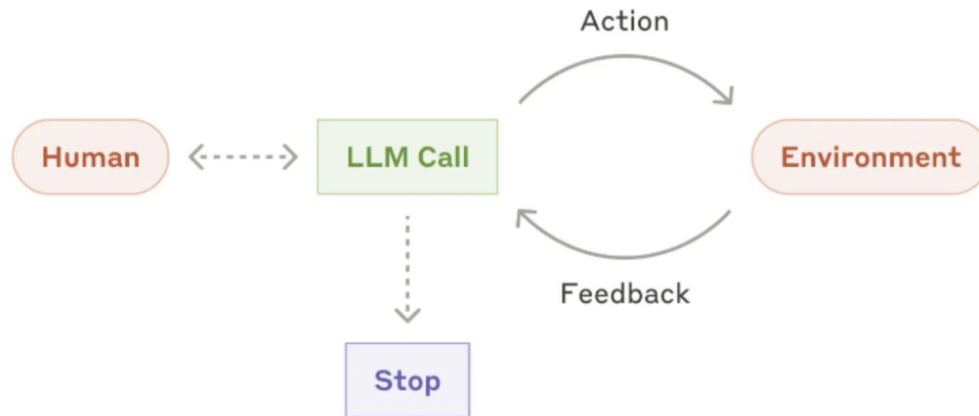
AI Agents



7

AI Agents

Autonomous loop — Agents plan, take actions via tool-calling, and respond to environmental feedback in a loop.



How they work

- Begin with a command or discussion from a human
- Plan and operate independently once task is clear
- Gain ground truth from environment at each step
- Pause for human feedback; stopping conditions maintain control

When to use & trade-offs

- Open-ended problems that can't be laid out as workflows
- Higher cost, potential for compounding errors
- Test in sandboxes with appropriate guardrails

Examples

- SWE-bench coding agent (multi-file edits)
- Computer-use reference implementation



Why Frameworks?

Core idea — Implementing these patterns does not require a framework. LangGraph aims to minimize the overhead — not replace your understanding.

1

Persistence

- Memory across runs
- Human-in-the-loop checkpoints

2

Streaming

- From any LLM call
- From any step in the workflow / agent

3

Deployment

- Testing
- Debugging
- Deploying

Frameworks reduce overhead — they don't replace understanding the underlying patterns.



Thank You!



Nunnari Academy

Generative AI & Agentic AI Program

Module: Agentic AI • Workflows & Agents